

Análisis de sistemas y requisitos en el desarrollo de software

Breve descripción:

Este componente integra el pensamiento sistémico para entender el software como un conjunto interrelacionado de personas, procesos y tecnología. Mediante la gestión de procesos y modelos como BPMN, se transforman entradas en soluciones de valor. Esto asegura el cumplimiento de requisitos funcionales (operación) y no funcionales (seguridad, desempeño) para garantizar la calidad.

Abril 2026

Tabla de contenido

Introducción	4
1. La Teoría General de Sistemas (TGS) y enfoque sistémico	7
1.1. Teoría General de Sistemas (TGS).....	7
1.2. Enfoque sistémico.....	12
2. Datos e información.....	17
2.1. Procesamiento de datos: concepto y modelo	19
3. Sistemas de Información (SI).....	24
4. Ciclo de vida del software (SDLC).....	30
4.1. SDLC vs. Metodología de desarrollo.....	30
4.2. Modelos de ciclo de vida	33
4.3. Metodologías de desarrollo de software	40
5. Procesos y gestión BPM	44
6. Requisitos	52
6.1. Requisitos funcionales y no funcionales.....	52
6.2. Ingeniería de requisitos	57
6.3. Técnicas de elicitación de requisitos	58
6.4. Fuentes de requisitos.....	60
7. Estadística descriptiva	62

7.1. Medidas de tendencia central.....	64
Síntesis	69
Glosario	70
Referencias bibliográficas	71
Créditos	72

Introducción

Este componente formativo invita a comprender el desarrollo y la calidad del software desde el pensamiento sistémico, reconociendo que el software es un sistema abierto en el que interactúan personas, procesos, datos y tecnología. Desde esta perspectiva, un error en el código no se interpreta como un hecho aislado, sino como el resultado de decisiones y procesos interrelacionados que deben analizarse y gestionarse de manera integral.

Es por ello, que, a lo largo del componente, el aprendiz fortalecerá competencias en ingeniería de requisitos, ciclos de vida y metodologías de desarrollo, gestión por procesos y técnicas de elicitación, así como en análisis cuantitativo mediante estadística descriptiva, orientado a la toma de decisiones informadas.

El objetivo es ir más allá de la simple detección de fallas, promoviendo una cultura de mejora continua y gestión del conocimiento que contribuya a la calidad, sostenibilidad y excelencia del servicio de software.

Partiendo de lo anterior, se invita a acceder al siguiente video, el cual contextualiza y articula la temática que se abordará durante este componente formativo:

Video 1. Análisis de sistemas y requisitos en el desarrollo de software



[Enlace de reproducción del video](#)

Transcripción del video: Análisis de sistemas y requisitos en el desarrollo de software

En el entorno tecnológico actual, comprender cómo se estructuran y funcionan los sistemas es esencial para diseñar soluciones que respondan a las necesidades de las organizaciones. Este recorrido formativo presenta una visión integrada sobre la relación entre sistemas, datos, procesos y desarrollo de software, elementos clave para la gestión de la información y la toma de decisiones.

A partir de los principios de la teoría general de sistemas y el enfoque sistémico, se analiza cómo los diferentes componentes de un sistema interactúan para transformar datos en información útil. Esta perspectiva permite entender que las soluciones tecnológicas forman parte de estructuras organizacionales interconectadas que procesan información y apoyan la gestión empresarial.

También se abordan los sistemas de información y el ciclo de vida del software, destacando la importancia de planificar, desarrollar y mantener soluciones tecnológicas de forma estructurada. En este contexto, se estudian los procesos organizacionales y su representación, así como la gestión de requisitos, elemento fundamental para identificar qué debe hacer un sistema y cómo debe responder a las necesidades del usuario.

Finalmente, se incorporan fundamentos de estadística descriptiva aplicada al análisis de datos, que permiten organizar, interpretar y representar información mediante medidas y gráficas. Estas herramientas fortalecen el análisis de métricas y la comprensión de datos en entornos tecnológicos.

De esta manera, el componente integra sistemas, procesos, requisitos y análisis de información, proporcionando bases conceptuales para comprender cómo se diseñan y gestionan soluciones tecnológicas en las organizaciones.

1. La Teoría General de Sistemas (TGS) y enfoque sistémico

La comprensión de fenómenos complejos exige miradas que vayan más allá del análisis fragmentado de sus partes. En este sentido, la Teoría General de Sistemas (TGS) y el enfoque sistémico ofrecen una perspectiva integradora que permite entender la realidad como un conjunto de elementos interrelacionados que interactúan de manera dinámica con su entorno. Ambos planteamientos coinciden en que el comportamiento de un sistema no puede explicarse únicamente desde sus componentes aislados, sino desde las relaciones, procesos y propósitos que los articulan.

Estos enfoques han demostrado su utilidad en múltiples campos del conocimiento, al proporcionar principios y herramientas para analizar, interpretar y gestionar la complejidad. En contextos organizacionales, educativos y tecnológicos — como el desarrollo de software —, permiten identificar patrones, anticipar efectos y tomar decisiones más coherentes y sostenibles. Con esta base conceptual, se abordan a continuación cada uno de ellos, profundizando en sus fundamentos, características y aplicaciones.

1.1. Teoría General de Sistemas (TGS)

Surgió como una respuesta crítica al reduccionismo científico dominante en la primera mitad del siglo XX, el cual buscaba explicar los fenómenos complejos, mediante el estudio aislado de sus partes. Este enfoque resultaba limitado para comprender realidades dinámicas e interrelacionadas.

El biólogo austriaco Ludwig Von Bertalanffy advirtió que, en los organismos vivos y en otros sistemas complejos, las propiedades esenciales emergen de las interacciones entre sus componentes y no de los elementos considerados de manera independiente. A partir de esta observación, propuso una visión integradora orientada al estudio de los sistemas como totalidades organizadas.

La TGS se concibió como un marco teórico interdisciplinario, con el propósito de identificar principios comunes aplicables a distintas áreas del conocimiento, sentando las bases del pensamiento sistémico como una nueva forma de comprensión científica.

Para tener mayor claridad en sus procesos evolutivos, a continuación, se presenta una cronología ampliada de la Teoría General de Sistemas (TGS), integrando las acciones previamente descritas y extendiéndola hasta desarrollos más actuales del pensamiento sistémico:

a) 1900–1930

Predomina el reduccionismo científico, que explica los fenómenos complejos a partir del análisis aislado de sus partes, especialmente en ciencias naturales y sociales.

b) 1937

El biólogo austriaco Ludwig Von Bertalanffy formula los primeros planteamientos de la TGS, cuestionando el enfoque reduccionista y proponiendo estudiar los sistemas como totalidades organizadas.

c) 1937–1945

Bertalanffy desarrolla los fundamentos teóricos iniciales de la TGS, aplicables a múltiples disciplinas como biología, física, psicología, economía e ingeniería.

d) Década de 1940

La TGS se fortalece con aportes de la cibernética (Norbert Wiener), introduciendo conceptos como retroalimentación, control y comunicación en sistemas.

e) 1950–1960

La teoría se difunde en ciencias sociales y administración. Surgen aplicaciones en organizaciones, educación y gestión, consolidando el enfoque interdisciplinario.

f) 1968

Publicación de General System Theory, obra cumbre de Bertalanffy, que formaliza la TGS y consolida el pensamiento sistémico como nuevo paradigma científico.

g) Décadas de 1970–1980

La TGS influye en la teoría de sistemas abiertos, la dinámica de sistemas (Jay Forrester) y el análisis organizacional.

h) Décadas de 1990–2000

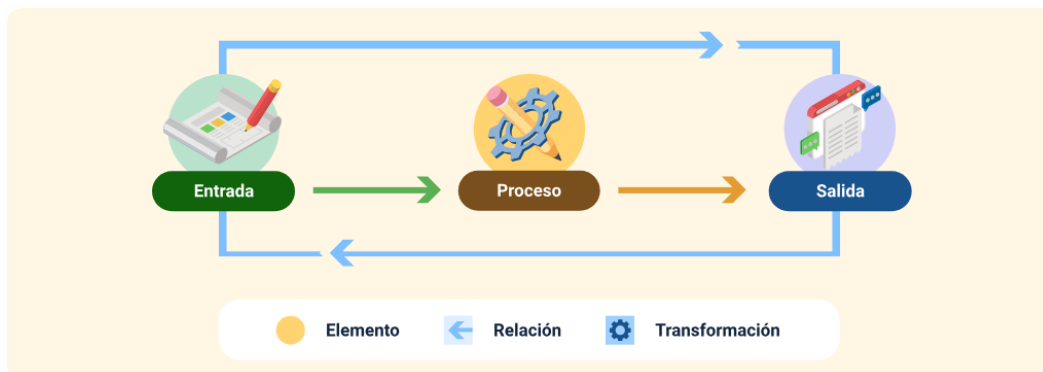
Evoluciona hacia enfoques de complejidad, autoorganización y sistemas adaptativos complejos, aplicados a tecnología, ecología y economía.

i) 2010–actualidad

El pensamiento sistémico se integra con la transformación digital, la ingeniería de software, la gestión del conocimiento y el análisis de sistemas socio-técnicos, manteniendo plena vigencia en contextos complejos e interconectados.

Complementando la anterior información, se presenta la siguiente imagen representativa:

Figura 1. Estructura básica de la TGS



“El todo es más que la suma de sus partes”

— Aristóteles, retomado por Bertalanffy como principio fundacional de la TGS.

Para complementar lo visto, es importante conocer la siguiente cronología de los orígenes de la Teoría General de Sistemas:

a) Antecedentes filosóficos

El holismo de Smuts y la Gestalt Psychology sientan las bases del pensamiento sistémico.

b) Primera formulación

Bertalanffy presenta la “Teoría de los sistemas abiertos” en la Universidad de Chicago.

c) Cibernética de Wiener

Wiener publica “Cybernetics” introduciendo la retroalimentación y el control en sistemas.

d) Sociedad para la investigación de sistemas generales

Bertalanffy y otros fundan la primera sociedad dedicada al estudio de sistemas.

e) Obra cumbre publicada

General System Theory consolida la TGS como disciplina científica independiente.

Conceptos esenciales de la TGS

La TGS establece un vocabulario común para describir cualquier sistema, independientemente de su naturaleza. Estos conceptos son la base del análisis sistémico y se aplican directamente al diseño, desarrollo y prueba de software.

Por lo anterior, a continuación, se presenta de manera conceptual los componentes o propiedades de un sistema:

- **Emergencia:** propiedades que surgen de las interacciones del sistema.
- **Entorno:** factores externos que influyen en el sistema.
- **Subsistemas:** componentes del sistema que pueden analizarse individualmente.
- **Retroalimentación:** mecanismo de autorregulación del sistema.
- **Homeostasis:** capacidad del sistema para mantener el equilibrio.

1.2. Enfoque sistémico

Es una manera de pensar y analizar la realidad que concibe cualquier fenómeno una organización, un proceso de software y un ecosistema como un sistema integrado, no como una suma de partes independientes. Surge directamente de la Teoría General de Sistemas (TGS) de Bertalanffy y se convirtió en el paradigma dominante para gestionar complejidad en el siglo XX y XXI.

A diferencia del pensamiento analítico clásico, que descompone los problemas en partes aisladas para estudiarlas por separado, el enfoque sistémico preserva las relaciones y las interacciones como objeto central de estudio. Su premisa fundamental es que la comprensión del todo requiere observar cómo sus partes se conectan, se influyen mutuamente y producen comportamientos emergentes que ninguna parte podría generar por sí sola.

En el ámbito del desarrollo de software, el enfoque sistémico permite comprender que un defecto detectado en producción no es un problema aislado de una función: es el síntoma de un sistema de procesos, decisiones y comunicación que falló en múltiples puntos interrelacionados.

- **Características**

El enfoque sistémico proporciona un marco conceptual para analizar y comprender realidades complejas desde una perspectiva integral. En lugar de fragmentar los problemas en partes aisladas, propone entender cómo interactúan los elementos dentro de un conjunto organizado, identificando patrones, relaciones y dinámicas que influyen en su comportamiento. A partir de esta base, se describen a

continuación las principales características que orientan el pensamiento y la intervención sistémica:

- **Visión holística**

Examina el sistema completo antes de analizar sus partes. El comportamiento global no se puede deducir de los componentes individuales.

- **Pensamiento circular**

Las relaciones causa-efecto son cíclicas, no lineales. Las causas retroalimentan sus propios efectos, creando bucles de refuerzo o equilibrio.

- **Interdependencia**

Cada elemento del sistema afecta y es afectado por los demás. Un cambio en una parte repercute en todo el sistema, a veces de forma inesperada.

- **Dinámica temporal**

Incorpora el tiempo como variable esencial. Los efectos de las decisiones pueden manifestarse con demoras que dificultan identificar las causas reales.

- **Modelado y simulación**

Utiliza modelos (diagramas causales, arquetipos sistémicos) para representar, comunicar y simular el comportamiento del sistema antes de intervenir.

- **Orientación al propósito**

Todo análisis sistémico parte de la función u objetivo del sistema. Las intervenciones buscan alinear la estructura interna con ese propósito.

- **Aplicación**

Aplicar el enfoque sistémico comprende un proceso metodológico con pasos concretos que guían desde la observación inicial hasta la intervención y el monitoreo de resultados:

a) Definir el sistema y sus límites

Identificar qué está dentro del sistema de análisis y qué pertenece al entorno. Delimitar los actores, procesos y recursos que interactúan. Por ejemplo: en software: ¿qué módulos, equipos y procesos forman parte del sistema de desarrollo?

b) Mapear las interrelaciones

Construir un diagrama causal que muestre cómo los elementos del sistema se influyen mutuamente. Identificar los bucles de retroalimentación: ¿cuáles amplifican el cambio (refuerzo) y cuáles lo amortiguan (equilibrio)?

c) Identificar patrones de comportamiento

Analizar el comportamiento del sistema a lo largo del tiempo: ¿crece linealmente, oscila, colapsa? Los arquetipos sistémicos (límites del crecimiento, transferencia de carga, escalada) permiten reconocer patrones recurrentes.

d) Encontrar los puntos de apalancamiento

Determinar dónde una intervención pequeña puede producir un cambio grande y sostenido. Generalmente no están donde parece obvio: cambiar reglas del sistema, flujos de información o paradigmas suele ser más poderoso que cambiar parámetros.

e) Intervenir, aprender y ajustar

Implementar los cambios de forma incremental, observar los efectos sistémicos (incluyendo las consecuencias no intencionadas) y retroalimentar el modelo con lo aprendido. El enfoque sistémico es iterativo por naturaleza.

- **Organizaciones inteligentes**

El concepto de organización inteligente (learning organization) fue desarrollado por Peter Senge en su obra “La Quinta Disciplina (1990)”, como la máxima expresión práctica del enfoque sistémico en el ámbito empresarial. Una organización inteligente es aquella que ha desarrollado la capacidad continua de aprender, adaptarse y transformarse frente a un entorno cambiante.

Senge argumenta que la mayoría de las organizaciones padecen discapacidades de aprendizaje — patrones mentales y estructurales que les impiden ver la realidad sistémica y aprender de ella. La solución no es contratar personas más inteligentes: es construir sistemas organizacionales que favorezcan el aprendizaje colectivo.

Las características de las organizaciones inteligentes son:

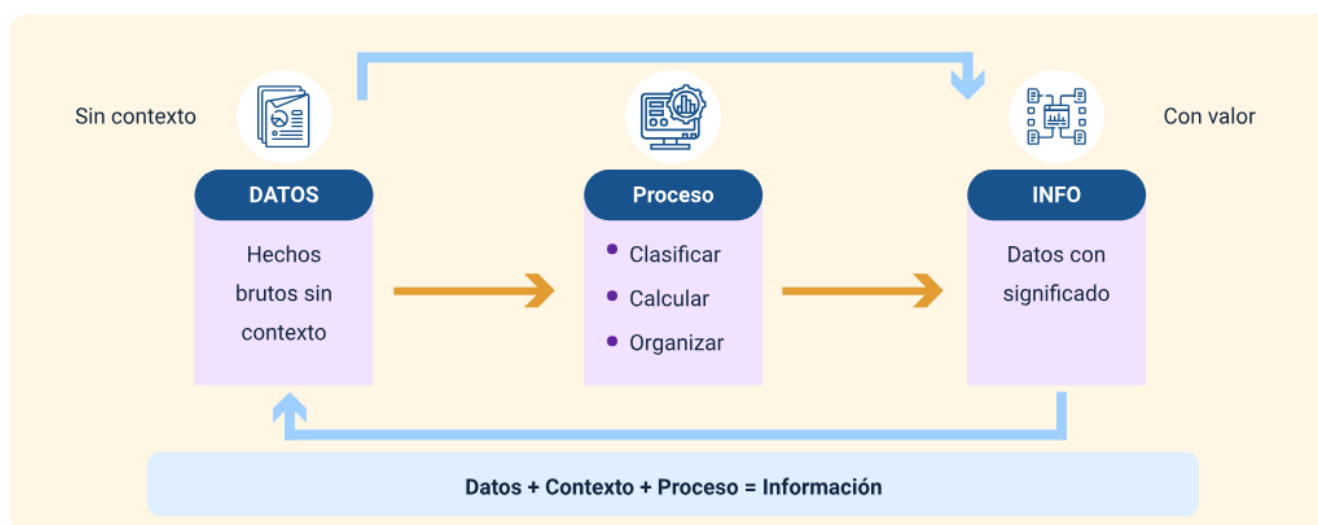
- **Aprendizaje continuo:** aprende de sus errores y aciertos de forma sistemática y colectiva.
- **Visión compartida:** todos los miembros comprenden y comparten el propósito y metas del sistema.
- **Trabajo en equipo:** el aprendizaje colectivo supera la suma del aprendizaje individual.

- **Modelos mentales:** cuestiona activamente los supuestos y creencias que guían las decisiones.
- **Dominio personal:** cada persona se compromete con su propio crecimiento y aprendizaje.
- **Pensamiento sistémico:** comprende las interrelaciones en lugar de reaccionar a eventos aislados.

2. Datos e información

En el campo de los sistemas de información y el desarrollo de software, los términos dato e información suelen usarse indistintamente en el lenguaje cotidiano, pero representan conceptos técnicamente distintos y complementarios. Comprender su diferencia es la base de todo procesamiento computacional. Antes de entrar en detalle en cada concepto es importante analizar el siguiente esquema:

Figura 2. Proceso de la información



Proceso de la información

Sin contexto

- DATOS
 - Hechos brutos sin contexto
- Proceso
 - Clasificar
 - Calcular

- Organizar
- INFO
 - Datos con significado
- Con valor

Datos + Contexto + Proceso = Información

A continuación, la explicación de cada uno y una ejemplificación para tener mayor claridad al respecto:

- **Dato**

Es un hecho, cifra, símbolo o señal sin interpretar, que describe un objeto, evento o condición. Por sí solo carece de significado contextual. Es la materia prima del procesamiento.

Ejemplo: 37.5 · Bogotá · TRUE · 11:42 · 0x3A

- **Información**

Es un conjunto de datos procesados, organizados e interpretados dentro de un contexto que les otorga significado, relevancia y propósito para quien la recibe.

Ejemplo: 37.5 °C = temperatura corporal con fiebre registrada en paciente a las 11:42 a.m. en Bogotá.

Igualmente, se detallan las siguientes características de cada acción:

Tabla 1. Comparativo dato e información

Característica	Dato	Información
Naturaleza	Bruto, sin interpretar, fragmentado.	Procesada, organizada, contextualizada.
Significado	Ninguno de manera aislada.	Tiene sentido para el receptor.
Dependencia	Independiente del contexto.	Depende del contexto y del propósito.
Ejemplo en software	status=0 en un log.	"El módulo de autenticación finalizó correctamente".
Forma	Números, letras, señales, bits.	Reportes, gráficas, mensajes, alertas.

Existen tipos de datos según su naturaleza como son:

- Numéricos (enteros, decimales).
- Alfanuméricos (texto).
- Lógicos (booleanos).
- Temporales (fecha, hora).
- Multimedia (imagen, audio, video).
- Geoespaciales (coordenadas).

2.1. Procesamiento de datos: concepto y modelo

El procesamiento de datos es el conjunto de operaciones sistemáticas que transforman datos brutos en información útil para la toma de decisiones. Es el núcleo funcional de cualquier sistema de información y el proceso que da sentido a los datos recolectados.

Desde una perspectiva sistémica, el procesamiento de datos opera bajo el modelo IPO (Input – Process – Output): los datos ingresan al sistema (entrada), son transformados mediante operaciones específicas (proceso) y se producen resultados con valor informativo (salida). A este ciclo se suma la retroalimentación, que permite verificar y mejorar el proceso. Dicho proceso, se realiza de manera completa de la siguiente manera:

- **Entrada:** captura y recolección de datos crudos desde fuentes internas o externas.
- **Validación:** verificación de integridad, formato y completitud de los datos recibidos.
- **Procesamiento:** clasificación, cálculo, ordenación y transformación de los datos.
- **Almacenamiento:** persistencia del resultado en bases de datos, archivos o repositorios.
- **Salida:** presentación de la información procesada: reportes, dashboards, alertas.

A. Actividades del procesamiento de datos

El procesamiento de datos no es una operación única, es un conjunto de actividades especializadas que pueden ejecutarse de forma secuencial o combinada, dependiendo del sistema de información y el propósito del procesamiento. Cada actividad agrega valor transformando los datos progresivamente de la siguiente manera:

a) Captura y recolección

Obtención de datos desde fuentes primarias (sensores, formularios, teclado, APIs, archivos) o secundarias (bases de datos externas, web scraping). Es el punto de entrada al ciclo de procesamiento.

Ejemplo: formulario de registro de usuarios, lectura de sensores IoT, importación de CSV.

b) Validación y verificación

Comprobación de que los datos cumplen reglas de integridad: formato correcto, valores dentro de rangos permitidos, campos obligatorios presentes, consistencia entre campos relacionados. Detecta errores antes de procesar.

Ejemplo: verificar que un email tiene formato válido, que la edad sea un número positivo menor a 150.

c) Clasificación y organización

Agrupación de datos en categorías predefinidas según atributos comunes. Permite estructurar la información para facilitar su recuperación, análisis y comparación posterior. Incluye la creación de taxonomías y jerarquías.

Ejemplo: clasificar transacciones por tipo (ingreso/egreso/transferencia) o clientes por segmento.

d) Cálculo y transformación

Aplicación de operaciones matemáticas, lógicas o algorítmicas para derivar nuevos valores a partir de los datos existentes. Es la actividad más computacionalmente intensiva y el corazón del procesamiento cuantitativa.

Ejemplo: calcular promedio de ventas, aplicar descuentos, convertir monedas, calcular métricas de cobertura.

e) Ordenamiento

Disposición de los datos en una secuencia específica (ascendente, descendente, cronológica, alfabética) según uno o múltiples criterios. Facilita la búsqueda, presentación y análisis comparativo de la información.

Ejemplo: ordenar defectos por severidad, listar usuarios por fecha de registro, ranking de ventas.

f) Fusión e integración

Combinación de datos provenientes de múltiples fuentes en una estructura unificada y coherente. Fundamental en entornos con múltiples sistemas y requiere resolver conflictos de formato, duplicados y contradicciones.

Ejemplo: cruzar datos de ventas con inventario y CRM para producir un reporte consolidado.

g) Almacenamiento y recuperación

Persistencia estructurada de los datos procesados en un medio de almacenamiento (base de datos relacional, NoSQL, data warehouse, archivos) y posterior recuperación eficiente mediante índices, claves y consultas.

Ejemplo: guardar resultados de pruebas en BD, indexar logs para consulta rápida por rango de fechas.

h) Resumen y presentación

Síntesis y visualización de los datos procesados en formatos comprensibles para el usuario final: tablas, gráficas, dashboards, reportes o alertas. Es la actividad que materializa la información y la hace accionable.

Ejemplo: dashboard de métricas de calidad, reporte ejecutivo de defectos, alertas automáticas por umbrales.

3. Sistemas de Información (SI)

Un Sistema de Información (SI) es un conjunto organizado e integrado de personas, procesos, tecnología y datos que trabajan conjuntamente para recopilar, almacenar, procesar y distribuir información que apoye la toma de decisiones, la coordinación, el control y el análisis dentro de una organización. A diferencia de un simple sistema informático, un SI no se reduce a hardware y software: incluye también los procedimientos, las personas que lo operan y el contexto organizacional en el que opera. Es la articulación entre la dimensión técnica y la dimensión humana de la gestión de información.

Desde la Teoría General de Sistemas, un SI es un sistema abierto que recibe datos del entorno (entradas), los transforma mediante procesos y reglas (proceso) y produce información útil (salidas), retroalimentándose continuamente para ajustar su funcionamiento a las necesidades de la organización.

- **Elementos constitutivos de los sistemas de información**

A continuación, se presenta un pódcast que profundiza en los conceptos abordados sobre los sistemas de información y sus elementos esenciales. Este recurso permite reforzar la comprensión del tema mediante una explicación clara y contextualizada, facilitando su consulta como apoyo al aprendizaje:

Transcripción del pódcast. Elementos constitutivos de los sistemas de información
Hombre: todas las organizaciones dependen de información para funcionar. Para operar, para planear, para decidir. Y esa información no aparece por arte de

Transcripción del pódcast. Elementos constitutivos de los sistemas de información

magia: hay una estructura que la produce, la organiza y la entrega a quien la necesita en el momento justo. A esa estructura la llamamos sistema de información.

Mujer: en este episodio vamos a ver de qué está hecho ese sistema. Cuáles son los elementos que lo conforman, cómo interactúan entre sí y por qué entenderlos cambia la forma en que uno trabaja con tecnología dentro de una organización.

Mujer: un sistema de información está conformado por seis elementos que interactúan de forma permanente. Lo que los hace poderosos es su interdependencia: cuando todos están alineados, la información fluye. Cuando alguno falla, ese flujo se interrumpe y la organización empieza a decidir con información incompleta, tardía o poco confiable.

Hombre: empecemos por las personas, que son el elemento más importante de los seis. Los usuarios que interactúan con el sistema a diario, los operadores que lo mantienen funcionando y los directivos que dependen de lo que produce para decidir. El sistema existe para ellos. Y cuando se diseña sin tenerlos en cuenta, la tecnología puede ser impecable y el sistema seguir siendo inútil.

Mujer: el segundo elemento son los procesos. Las reglas y procedimientos que definen cómo se captura la información, cómo se procesa, cómo se almacena y cómo llega a quien la necesita. Son el esqueleto que le da forma y coherencia al sistema. Una organización puede tener la mejor tecnología del mercado y aun así producir información caótica si los procesos que la soportan no están bien definidos.

Hombre: y el tercero son los datos. La materia prima de todo el sistema. Lo que entra determina lo que sale, y si lo que entra está incompleto, desactualizado o

Transcripción del pódcast. Elementos constitutivos de los sistemas de información

simplemente mal registrado, la información que produce el sistema arrastra ese error sin que nadie lo note hasta que una decisión sale mal.

Mujer: los tres elementos restantes son los que hacen posible que todo lo anterior ocurra. El hardware es la infraestructura física: servidores, computadoras, dispositivos de red, medios de almacenamiento. Es la plataforma sobre la que corre el sistema. Sin ella, el software más sofisticado no tiene dónde ejecutarse.

Hombre: el software son los programas y aplicaciones que implementan la lógica del negocio y transforman los datos en información útil. Es el elemento más visible para el usuario porque es lo que toca y lo que ve. Y el que más frecuentemente recibe la culpa cuando algo falla, aunque el problema real muchas veces esté en otro elemento del sistema.

Mujer: y las redes y comunicaciones son la infraestructura que permite que los datos viajen entre los componentes del sistema y lleguen a quienes los necesitan. Es el elemento más ignorado cuando funciona y el más notado cuando falla, porque su ausencia paraliza todo lo demás de forma inmediata.

Hombre: personas, procesos, datos, hardware, software y redes. Seis elementos que trabajan juntos para que una organización pueda decidir con información real.

Mujer: porque al final, un sistema de información bien construido no se nota. Simplemente funciona. Y detrás de cada decisión acertada que toma una organización, ahí está, invisible y esencial, haciendo su trabajo.

Transcripción del podcast. Elementos constitutivos de los sistemas de información
--

Hombre: gracias por su compañía. Nos escuchamos en el próximo programa.
--

- **Apoyo a la toma de decisiones**

Un Sistema de Información (SI) tiene como propósito fundamental proveer a directivos y gerentes información oportuna, relevante y precisa que sirva de base para la toma de decisiones estratégicas, tácticas y operativas, reduciendo la incertidumbre y el riesgo asociados a la gestión organizacional. Además, contribuye a la automatización y eficiencia operacional al disminuir tiempos, costos y errores humanos en procesos rutinarios, lo que permite que el talento humano se concentre en actividades de mayor valor agregado y aumente la productividad global.

De igual manera, un (SI) fortalece la coordinación y comunicación organizacional al facilitar el flujo de información entre departamentos, niveles jerárquicos y actores externos, eliminando silos informativos y promoviendo la alineación institucional. También posibilita el control y seguimiento del desempeño mediante indicadores clave (KPI), alertas tempranas y monitoreo del cumplimiento de metas. Finalmente, cuando se gestiona estratégicamente, se convierte en una fuente de ventaja competitiva y creación de valor, al permitir la personalización de servicios, la optimización de procesos, la innovación continua y la toma de decisiones basada en datos.

- **Clasificación de los sistemas de información**

Los sistemas de información se clasifican según el nivel organizacional al que sirven, el tipo de decisiones que apoyan y la naturaleza de su función en la

organización. Se clasifican principalmente según el nivel organizacional al que sirven y el tipo de soporte que brindan. Esta clasificación, propuesta por Laudon & Laudon (2020) y complementada por otros autores, es la más utilizada en la academia y en la industria.

- **Pirámide organizacional:** cada tipo de SI atiende un nivel diferente de la organización: desde los trabajadores operativos hasta la alta dirección. Cuanto más alto el nivel, más sintética y estratégica es la información que se produce.

La siguiente tabla, relaciona cada uno de estos niveles de dicha pirámide y sus respectivos procesos:

Tabla 2. Niveles pirámide organizacional

Nivel	Sistema	Sigla	Función principal	Usuarios típicos
Estratégico	Sistema de soporte a ejecutivos.	ESS/EIS	Proveer información resumida y tendencias para decisiones de largo plazo.	Alta dirección, CEO, junta directiva.
Directivo	Sistema de información gerencial.	MIS	Generar reportes periódicos de gestión para control y planificación táctica.	Gerentes de área, directores funcionales.
Directivo	Sistema de soporte a decisiones.	DSS	Modelar escenarios, analizar datos y evaluar alternativas de decisión.	Analistas, gerentes de nivel medio.
Conocimiento	Sistema de trabajo de conocimiento.	KWS	Apoyar la creación y aplicación de conocimiento especializado.	Ingenieros, arquitectos, científicos, abogados.

Nivel	Sistema	Sigla	Función principal	Usuarios típicos
Conocimiento	Sistema de automatización de oficinas.	OAS	Gestionar documentos, comunicaciones y colaboración del conocimiento.	Trabajadores de oficina, secretarías, asistentes.
Operativo	Sistema de procesamiento de transacciones.	TPS	Registrar y procesar las transacciones diarias del negocio en tiempo real.	Cajeros, operadores, vendedores, personal operativo.

- **Aplicación en el SENA**

El SENA utiliza varios SI integrados: el sistema de gestión académica (tipo TPS/MIS) para matrículas y notas, la plataforma SOFIA Plus (KWS/OAS) para contenidos de aprendizaje, y sistemas administrativos para presupuesto y contratación (ERP).

4. Ciclo de vida del software (SDLC)

Es el conjunto estructurado de fases, actividades, tareas y entregables que describen el proceso completo de creación, despliegue, mantenimiento y eventual retiro de un sistema de software, desde la concepción de la idea hasta su discontinuación.

El SDLC no es un proceso único y universal, existen múltiples modelos que organizan estas fases de formas distintas según la naturaleza del proyecto, el nivel de incertidumbre en los requisitos, el tamaño del equipo y las restricciones de tiempo y presupuesto. Lo que todos comparten es la misma finalidad: producir software de calidad de forma controlada, predecible y repetible.

La importancia del SDLC para los equipos de pruebas y calidad (QA) es fundamental: el modelo de ciclo de vida adoptado determina cuándo, cómo y en qué profundidad se integrará la actividad de testing en el proyecto. Un tester que comprende el SDLC puede anticipar sus intervenciones, diseñar mejor los artefactos de prueba y comunicarse efectivamente con el equipo de desarrollo.

4.1. SDLC vs. Metodología de desarrollo

El ciclo de vida del desarrollo de software (SDLC) establece qué fases componen el proceso y en qué secuencia se desarrollan, mientras que la metodología define cómo se ejecutan dichas fases; es decir, qué prácticas, roles, ceremonias y herramientas se emplean en su implementación. Por ejemplo, Scrum es una metodología que aplica un enfoque iterativo e incremental del SDLC.

Cada fase del ciclo constituye un período con un propósito específico, actividades claramente definidas y criterios de entrada y salida que determinan su inicio y cierre. Dentro de estas fases se realizan actividades o tareas concretas como el análisis de requisitos, el diseño de arquitectura, la codificación, la ejecución de pruebas y el despliegue, las cuales generan entregables tangibles —como el documento de requisitos (SRS), diseños técnicos, código fuente, planes y reportes de pruebas o manuales de usuario— que sirven de insumo para la siguiente etapa.

El proceso se sustenta, además, en criterios de calidad que permiten aprobar cada fase y garantizar que se cumplan los estándares establecidos antes de avanzar, así como en la definición clara de roles y responsabilidades denominados product owner, arquitecto, desarrollador, tester, líder de proyecto, usuario final o auditor de calidad que aseguran una adecuada distribución del trabajo. Complementariamente, se incorporan mecanismos de control como revisiones, auditorías, métricas e indicadores para monitorear el avance, identificar desviaciones y aplicar acciones correctivas. Aunque cada modelo organiza el proceso de manera particular, la mayoría coincide en un conjunto de fases fundamentales que constituyen el núcleo del desarrollo formal de software.

A continuación, se presenta una síntesis de las fases fundamentales, describiendo el objetivo principal de cada etapa, sus entregables clave y el rol específico de aseguramiento de la calidad (QA). Esta visión permite comprender cómo la calidad se integra de manera transversal a lo largo de todo el proceso de desarrollo:

Tabla 3. Fases SDLC

#	Fase	Objetivo principal	Entregable clave	Rol de QA
1	Planificación	Definir el alcance, viabilidad, cronograma, recursos y riesgos del proyecto antes de iniciar el desarrollo.	Plan de proyecto, análisis de viabilidad.	Revisión de riesgos de calidad; estimación del esfuerzo de pruebas.
2	Análisis de requisitos	Elicitar, documentar y validar las necesidades funcionales y no funcionales del sistema con los stakeholders.	SRS (especificación de requisitos).	Revisión de requisitos para detectar ambigüedades; preparar casos de prueba de aceptación.
3	Diseño del sistema	Definir la arquitectura, los componentes, las interfaces, los datos y la infraestructura del sistema.	Documento de diseño, diagramas UML.	Revisión del diseño; diseño de pruebas de integración y de sistema.
4	Implementación	Codificar los componentes del sistema siguiendo los estándares y el diseño aprobado.	Código fuente, componentes integrados.	Pruebas unitarias; code review; análisis estático de código.
5	Pruebas	Verificar que el sistema cumple los requisitos funcionales y no funcionales; detectar y reportar defectos.	Reporte de pruebas, registro de defectos.	Ejecución de pruebas funcionales, de rendimiento, seguridad e integración.
6	Despliegue	Instalar, configurar y poner en operación el sistema en el entorno de producción para los usuarios finales.	Sistema en producción, manual de usuario.	Pruebas de aceptación del usuario (UAT); smoke testing en producción.

#	Fase	Objetivo principal	Entregable clave	Rol de QA
7	Mantenimiento	Corregir defectos post-producción, adaptar el sistema a cambios del entorno y agregar nuevas funcionalidades.	Parches, actualizaciones, releases.	Pruebas de regresión; pruebas de impacto de cambio.

- **El rol de QA no comienza en la fase de pruebas**

Un error frecuente es pensar que el trabajo del tester inicia en la fase 5. En realidad, el aseguramiento de calidad interviene desde la fase Planificación: revisar requisitos, inspeccionar el diseño y realizar pruebas unitarias durante la implementación reduce drásticamente el costo de corrección de defectos (principio de detección temprana del IEEE 730).

4.2. Modelos de ciclo de vida

Un modelo de ciclo de vida es una representación abstracta del proceso de desarrollo que describe cómo se organizan, secuencian y relacionan las fases del SDLC. La elección del modelo adecuado es una decisión estratégica que impacta directamente la calidad, el costo y el tiempo del proyecto. Por tal motivo, se detallan cada uno de ellos:

- **Modelo en cascada (Waterfall)**

Figura 3. Modelo en cascada



Modelo en cascada

- Flujo secuencial – cada fase inicial al completarse la anterior.
 - Planificación – Requisitos – Diseño – Implementación – Pruebas – Despliegue – Mantenimiento.

Es el modelo más antiguo y conocido, propuesto por Royce en 1970. Las fases se ejecutan de forma estrictamente secuencial: cada una debe completarse y aprobarse formalmente antes de iniciar la siguiente. No hay solapamiento ni retroceso planificado entre fases.

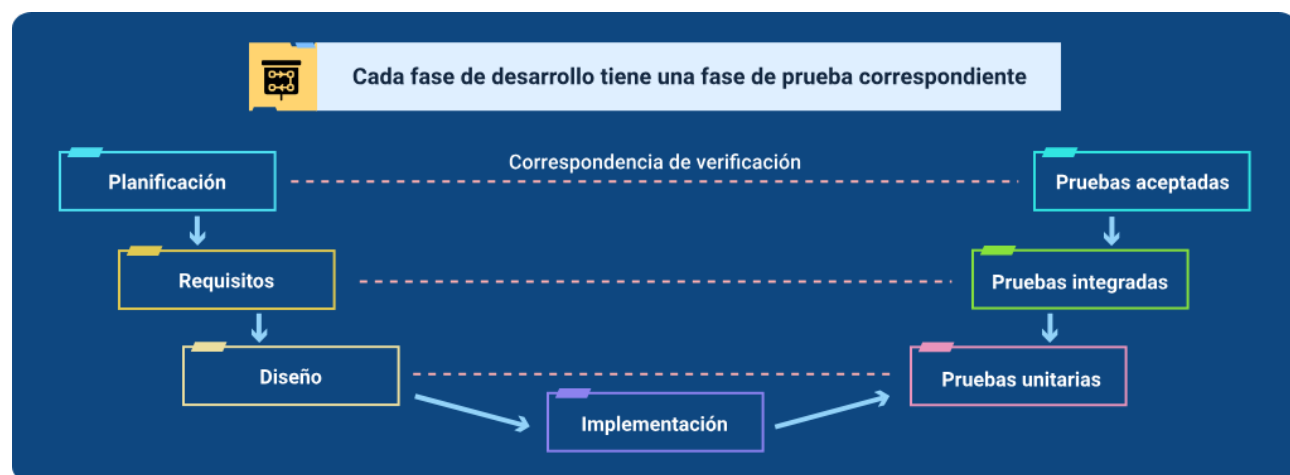
Se usa en proyectos con requisitos muy estables y bien definidos desde el inicio, contratos de precio fijo, sistemas críticos con alta necesidad de documentación formal (defensa, aeronáutica, sector bancario). Si los requisitos cambian a mitad del proyecto, el costo de retroceder es muy alto; por ello, si las pruebas ocurren tarde en el ciclo, se eleva el costo de corrección de defectos.

Entre sus principales ventajas se encuentran la facilidad para gestionar y documentar el proyecto, gracias a una estructura clara y ordenada, así como la

definición precisa de entregables en cada etapa, lo que facilita el control y seguimiento. Además, resulta especialmente adecuado para equipos grandes, donde la planificación detallada y la formalización de procesos favorecen la coordinación. Sin embargo, presenta desventajas importantes, como su rigidez frente a cambios en los requisitos, la realización tardía de pruebas, lo que puede generar defectos más costosos de corregir y el hecho de que el cliente solo visualiza el producto final al término del proceso, limitando la retroalimentación temprana.

- **Modelo en V (V-Model) verificación**

Figura 4. Modelo en V



Modelo en V

- Cada fase de desarrollo tiene una fase de prueba correspondiente

Correspondencia de verificación

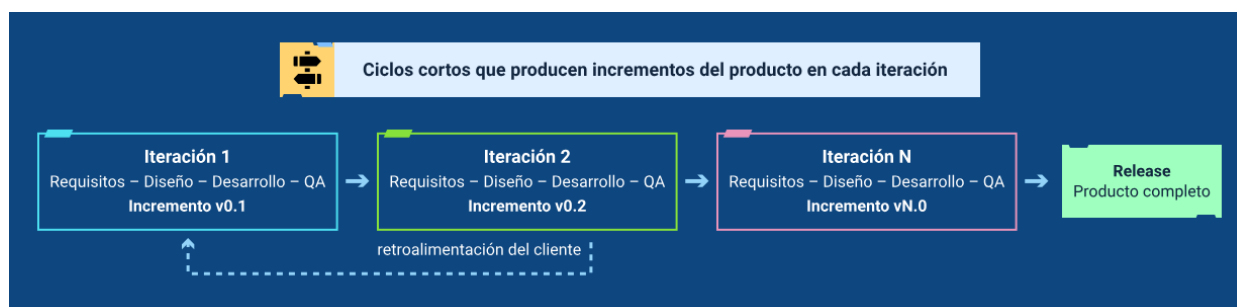
- Planificación – Requisitos – Diseño – Implementación – Pruebas unitarias – Pruebas integradas – Pruebas aceptadas

Este modelo es una extensión del modelo en cascada en la que cada fase del desarrollo tiene asociada una fase de prueba correspondiente. Gráficamente, la rama izquierda de la “V” representa las etapas de análisis y diseño, mientras que la rama derecha corresponde a los niveles de verificación y validación; en la base se encuentra la codificación, que conecta ambas dimensiones. Su principio fundamental es que las pruebas no se dejan para el final, sino que se planifican en paralelo al desarrollo: al definir los requisitos se diseñan las pruebas de aceptación, y al estructurar la arquitectura del sistema se preparan las pruebas de integración y sistema. Es especialmente recomendable en contextos donde la verificación formal es crítica — como en los sectores médico, financiero o aeronáutico— y suele emplearse junto con estándares como el IEEE 829.

Entre sus principales ventajas se encuentran la planificación temprana de pruebas, la alta trazabilidad entre requisitos y casos de prueba, así como la posibilidad de detectar defectos en fases iniciales. No obstante, mantiene una estructura relativamente rígida frente a cambios en los requisitos, no favorece el prototipado temprano y exige una amplia producción documental, lo que puede aumentar tiempos y costos en entornos dinámicos.

- **Modelo iterativo e incremental**

Figura 5. Modelo iterativo



Modelo iterativo

Ciclos cortos que producen incrementos del producto en cada iteración

- **Iteración 1**
 - Requisitos – Diseño – Desarrollo – QA
 - Incremento v0.1
- **Iteración 2**
 - Requisitos – Diseño – Desarrollo – QA
 - Incremento v0.2
- **Iteración N**
 - Requisitos – Diseño – Desarrollo – QA
 - Incremento vN.0
- **Release**
 - Producto completo

El sistema se construye en ciclos cortos (iteraciones), cada uno produciendo un incremento funcional del producto. Al final de cada iteración hay un entregable parcial que puede ser evaluado por el cliente, permitiendo incorporar retroalimentación antes de la siguiente iteración.

Base de metodologías ágiles: Scrum, XP y Kanban son implementaciones del modelo iterativo. La diferencia entre iterativo e incremental está en el énfasis: el iterativo refina el producto completo; el incremental añade módulos nuevos en cada ciclo.

Entre sus principales ventajas se destacan la posibilidad de obtener retroalimentación temprana del cliente, lo que permite validar el producto desde etapas iniciales; su alta adaptabilidad a cambios en los requisitos, facilitando ajustes conforme evoluciona el proyecto; y la integración del aseguramiento de la calidad (QA) en cada iteración, lo que contribuye a detectar y corregir errores de manera continua. No obstante, también presenta desventajas importantes: puede generarse scope creep (crecimiento descontrolado del alcance) si no existe una adecuada gestión de cambios; requiere una participación activa y constante del cliente, lo cual no siempre es viable; y puede resultar complejo estimar el costo total del proyecto desde el inicio debido a la naturaleza evolutiva del desarrollo. Además, existen otros modelos relevantes que ofrecen enfoques alternativos según las características y necesidades del proyecto.

De igual manera y no menos importantes, existen los siguientes modelos:

- **Modelo espiral**

Combina el enfoque iterativo con el énfasis en gestión de riesgos. En cada vuelta de la espiral se evalúan los riesgos antes de avanzar. Boehm (1988).

Ideal para proyectos grandes con alta incertidumbre técnica.

- **Metodologías ágiles**

Grupo de enfoques (Scrum, XP, Kanban) basados en el Manifiesto Ágil (2001): individuos sobre procesos, software funcional sobre documentación extensa, colaboración sobre contratos.

Ideal para startups y productos digitales con requisitos cambiantes.

- **Modelo DevOps**

Integra desarrollo, pruebas y operaciones en un ciclo continuo (CI/CD). El despliegue puede ser diario o incluso por horas y las pruebas automatizadas son su pilar fundamental.

Ideal para plataformas SaaS o servicios en la nube.

¿Qué modelo elegir?

Para tener mayores bases sobre esta importante respuesta, se presenta la siguiente tabla orientadora:

Tabla 4. Guía de decisión del modelo a elegir

Modelo	Requisitos	Cambios	Pruebas	Mejor para
Cascada	Estables y completos.	Muy difícil.	Fase tardía.	Contratos fijos, sistemas críticos documentados.
Modelo V	Estables.	Difícil.	Planificada desde el inicio.	Sistemas médicos, financieros, aeronáuticos.
Iterativo	Parcialmente conocidos.	Posible entre iteraciones.	Integrada en cada sprint.	Productos con cliente disponible y requisitos evolutivos.
Espiral	Inciertos o complejos.	Gestionado por riesgos.	En cada ciclo con análisis de riesgos.	Proyectos grandes con alta incertidumbre técnica.
Ágil	Cambiantes.	Bienvenidos.	Continua — TDD / BDD.	Startups, apps digitales, equipos pequeños y medianos.
DevOps	Muy cambiantes.	Continuos.	Automatizada — CI/CD.	Plataformas SaaS, microservicios, entrega continua.

4.3. Metodologías de desarrollo de software

Esta metodología es un marco estructurado de principios, prácticas, procesos y herramientas que orientan la forma en que un equipo organiza y ejecuta su trabajo para construir soluciones tecnológicas de manera sistemática, repetible y predecible.

A diferencia del modelo de ciclo de vida como se mencionó previamente que establece qué fases componen el proceso (análisis, diseño, desarrollo, pruebas,

implementación, etc.), la metodología define cómo se desarrollan esas fases en la práctica: qué roles intervienen, cómo fluye la comunicación, qué artefactos se generan, con qué frecuencia se realizan entregas y bajo qué criterios se toman decisiones técnicas y de gestión.

La selección de una metodología adecuada no es arbitraria, sino que depende de variables estratégicas como el tamaño y la distribución del equipo, el nivel de claridad y estabilidad de los requisitos, la complejidad y naturaleza del producto, la cultura organizacional y las expectativas del cliente. En este sentido, no existe una metodología universalmente superior, sino enfoques más o menos pertinentes según el contexto del proyecto.

Existen metodologías tradicionales predictivas como son:

a) RUP

Iterativa, centrada en arquitectura, casos de uso y cuatro fases (inicio, elaboración, construcción, transición), su mejor uso es en proyectos medianos-grandes con requisitos parcialmente definidos.

b) MSF - Microsoft Solutions Framework

Marco ágil con roles definidos, orientado al desarrollo de soluciones tecnológicas, especialmente en entornos corporativos con tecnologías Microsoft.

c) CMMI - Modelo de madurez

No es una metodología sino un marco de mejora de procesos en cinco niveles, su mejor uso es en organizaciones que buscan certificación y mejora continua de procesos.

d) Cascada -Secuencial estricta

Cada fase es aprobada formalmente antes de avanzar, su mejor uso es en contratos de precio fijo, requisitos estables y sistemas críticos

Igualmente, existen metodologías ágiles adaptativas:

- **Scrum**

Organiza el trabajo en sprints de 1–4 semanas. Define 3 roles (product owner, scrum master, development team), 3 artefactos (product backlog, sprint backlog, incremento) y 4 ceremonias (planning, daily, review, retrospective). Es el más usado a nivel mundial.

- **Extreme Programming (XP)**

Enfatiza las prácticas técnicas de excelencia: TDD (Test-Driven Development), programación en parejas, integración continua, refactoring y releases frecuentes. Diseñada para proyectos con requisitos muy cambiantes y complementa perfectamente a Scrum en la parte técnica.

- **Kanban**

Visualiza el trabajo en un tablero de columnas (por hacer / en progreso / hecho) y limita el trabajo en curso (WIP) para evitar sobrecarga. No tiene iteraciones fijas: el flujo es continuo, convirtiéndose en ideal para equipos de soporte, mantenimiento y operaciones.

- **Crystal, SAFe, LeSS y DevOps: Crystal**

Familia de metodologías adaptadas al tamaño del equipo.

- **SAFe**: Scrum escalado para grandes organizaciones (100+ personas).
- **LeSS**: Scrum para múltiples equipos.
- **DevOps**: integra desarrollo, pruebas y operaciones en un pipeline CI/CD continuo.

5. Procesos y gestión BPM

Un proceso es una secuencia estructurada de actividades interrelacionadas que transforman entradas (recursos, información, materiales) en salidas de valor (productos, servicios, información procesada), consumiendo recursos y siguiendo reglas definidas para alcanzar un objetivo específico.

En el contexto de la ingeniería de software y los sistemas de información, un proceso define el cómo se realiza el trabajo: qué actividades se ejecutan, en qué orden, quién las realiza, qué recursos consumen y qué resultados producen. Todo sistema organizacional se puede describir como un conjunto de procesos interconectados.

a) Características de un proceso

Antes de analizar los elementos que lo conforman, es clave entender que esta forma de trabajo permite ordenar las actividades, asegurar coherencia en las acciones y orientar los esfuerzos hacia resultados claros y medibles. Su correcta definición facilita el control, la mejora continua y la generación de valor tanto a nivel interno como externo. Por este motivo, se presentan las características esenciales que lo describen:

- **Tiene un objetivo definido**

Todo proceso existe para lograr un resultado específico y medible. Sin objetivo claro, no hay proceso, hay caos de actividades.

- **Interrelaciona actividades**

Las actividades del proceso están conectadas en una secuencia lógica donde la salida de una alimenta la entrada de la siguiente.

- **Tiene entradas y salidas**

Recibe insumos (datos, materiales, solicitudes) y produce resultados de valor para un cliente interno o externo.

- **Es medible y controlable**

Se pueden definir indicadores (KPI) para medir su eficiencia, calidad y tiempo. Lo que no se mide, no se puede mejorar.

- **Es repetible y replicable**

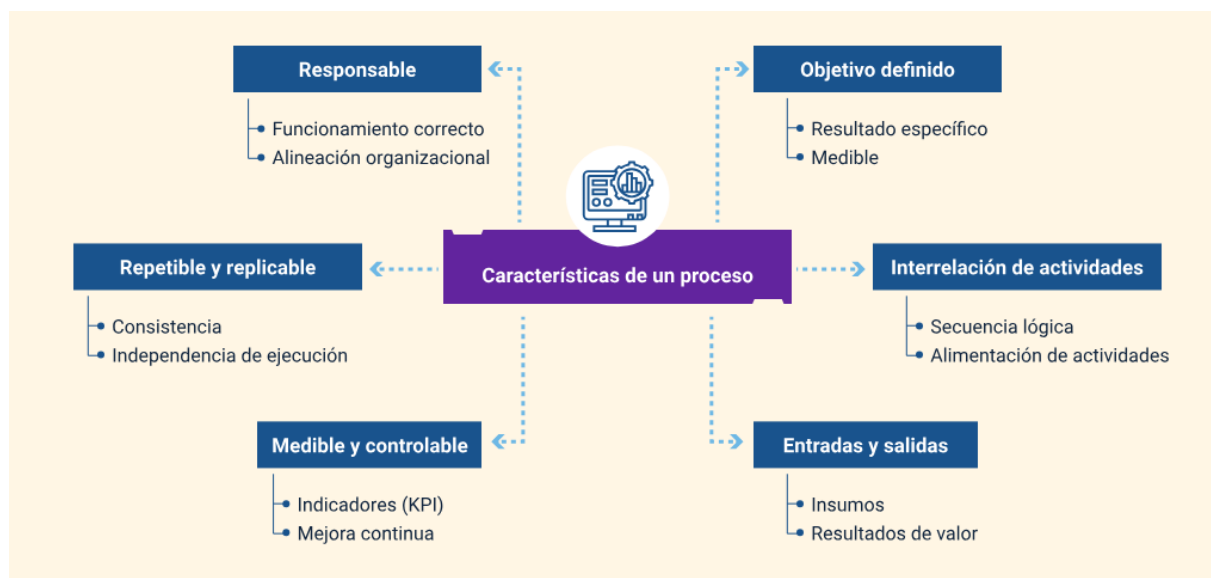
Un proceso bien definido produce resultados consistentes cada vez que se ejecuta, independientemente de quién lo realice.

- **Tiene un responsable**

Cada proceso tiene un dueño (process owner) que asegura su correcto funcionamiento, mejora continua y alineación con los objetivos organizacionales.

Las anteriores características, se representan de manera puntual en el siguiente esquema:

Figura 6. Características de un proceso



Características de un proceso

- Objetivo definido
 - Resultado específico
 - Medible
- Interrelación de actividades
 - Secuencia lógica
 - Alimentación de actividades
- Entradas y salidas
 - Insumos
 - Resultados de valor
- Medible y controlable
 - Indicadores (KPI)
 - Mejora continua
- Repetible y replicable

- Consistencia
- Independencia de ejecución
- Responsable
 - Funcionamiento correcto
 - Alineación organizacional

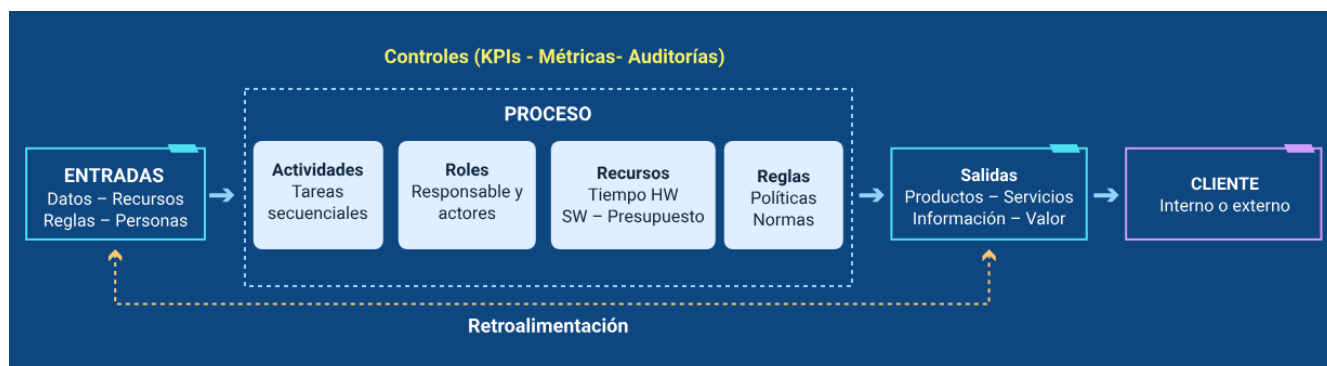
b) Tipos de entradas (inputs)

Las entradas son todos los recursos, información y condiciones que el proceso necesita para poder ejecutarse. Sin entradas adecuadas, el proceso no puede producir salidas de calidad (principio GIGO). Dentro de ellas se encuentran:

- **Información y datos:** documentos y registros que guían la ejecución del proceso.
- **Materiales / recursos:** software y suministros físicos necesarios para el proceso.
- **Recursos humanos:** personas con habilidades y conocimientos que ejecutan el proceso.
- **Reglas y restricciones:** políticas y normativas que condicionan la ejecución del proceso.

Al respecto, el siguiente esquema detalla los componentes de un proceso:

Figura 7. Componentes de un proceso



Componentes de un proceso

Controles (KPIs – Métricas – Auditorías)

- Entradas:
 - Datos – Recursos – Reglas – Personas
- Proceso
 - **Actividades:** tareas secuenciales
 - **Roles:** responsable y actores
 - **Recursos:** tiempo HW SW – Presupuesto
 - **Reglas:** políticas, normas
 - **Salidas:** productos – Servicios – Información – Valor
 - **Cliente:** interno o externo

c) Procedimientos: el detalle operativo del proceso

Un procedimiento es la descripción detallada y paso a paso de cómo ejecutar una actividad específica dentro de un proceso. Si el proceso dice "validar los datos del

usuario", el procedimiento describe exactamente cómo hacerlo: qué campos verificar, en qué orden, qué hacer ante cada tipo de error y quién aprueba el resultado.

La siguiente tabla, detalla los elementos presentes dentro de cada procedimiento:

Tabla 5. Elementos del procedimiento

Elemento del procedimiento	Descripción	Ejemplo en software
Objetivo	Qué resultado específico se busca alcanzar con el procedimiento.	Verificar que un caso de prueba cumple todos los criterios de aceptación.
Alcance	A qué actividades, sistemas o equipos aplica el procedimiento.	Aplica a todos los casos de prueba de regresión del módulo de autenticación.
Roles y responsabilidades	Quién ejecuta, quién aprueba, quién es notificado en cada paso.	Tester ejecuta; QA lead revisa; product owner aprueba resultado final.
Pasos secuenciales	La descripción numerada de cada acción a realizar en orden.	1) Abrir TestRail 2) Seleccionar ciclo 3) Ejecutar caso 4) Registrar resultado.
Criterios de aceptación	Las condiciones que determinan si el procedimiento fue exitoso.	El 100 % de los casos de alta prioridad deben pasar sin defectos críticos abiertos.
Registros y evidencias	Qué documentos o capturas se generan como evidencia de la ejecución.	Capturas de pantalla, log de ejecución, reporte de defectos en Jira.

d) Formas de representar gráficamente un proceso

La representación gráfica transforma la descripción textual de un proceso en un diagrama visual que facilita su comprensión, análisis y comunicación entre diferentes

actores (técnicos, gerentes, usuarios). Existen varios tipos de diagramas según el nivel de detalle y el propósito:

- **Diagrama de flujo**

Es el más tradicional. Usa formas geométricas estandarizadas (rectángulos, rombos, óvalos) para representar actividades, decisiones e inicio/fin. Ideal para procedimientos simples.

- **Diagrama BPMN - Business Process Model and Notation**

Es el estándar internacional (OMG) para modelar procesos de negocio. Rica notación con eventos, actividades, compuertas y flujos. Compatible con herramientas de automatización

- **Diagrama de carril (swimlane)**

Extiende el diagrama de flujo añadiendo carriles horizontales o verticales que representan los actores o departamentos responsables de cada actividad. Muestra claramente quién hace qué.

Partiendo de lo anterior, se relacionan las partes que se implementan en los diferentes diagramas elegidos y su respectivo significado:

- **Evento de inicio:** círculo delgado. Inicia el flujo del proceso. Puede ser disparado por tiempo, mensaje o condición.
- **Evento de fin:** círculo grueso. Marca la terminación del proceso o de un flujo. Puede ser final o intermedio.
- **Actividad / tarea:** rectángulo redondeado. Unidad de trabajo que realiza un actor. Puede ser manual, automática o de servicio.

- **Compuerta exclusiva:** rombo con X. Punto de decisión donde solo un camino es válido según la condición evaluada (sí/no).
- **Compuerta paralela:** rombo con +. Divide el flujo en múltiples caminos que se ejecutan simultáneamente (fork/join).
- **Flujo de secuencia:** flecha sólida. Conecta elementos y define el orden de ejecución del proceso.
- **Flujo de mensaje:** línea discontinua. Representa la comunicación entre participantes de diferentes pools o sistemas.
- **Pool / Lane:** contenedor que agrupa actividades de un participante. Los lanes dividen el pool por roles o departamentos.

6. Requisitos

Un requisito es una condición o capacidad que el sistema de software debe cumplir para satisfacer una necesidad del usuario o alcanzar un objetivo definido por las partes interesadas. Los requisitos son la base contractual entre el cliente y el equipo de desarrollo.

6.1. Requisitos funcionales y no funcionales

Los requisitos funcionales “el qué hace el sistema” constituyen la esencia operativa del software; definen las tareas específicas que el sistema debe realizar para satisfacer las necesidades del negocio. Se centran en el "qué" hace la solución, abarcando desde el procesamiento de cálculos automáticos y la generación de informes hasta la validación de entradas de datos para evitar errores de tipeo. En el contexto de la calidad del producto, un requisito funcional se considera exitoso cuando el sistema responde con exactitud a los requerimientos gerenciales en el tiempo estipulado, transformando datos desorganizados en información útil y veraz.

Se expresan típicamente como: "El sistema debe permitir..." / "El sistema deberá..." / "Como [rol], se quiere [acción] para [beneficio]". Son la base directa de los casos de prueba funcionales.

Las siguientes, son las categorías de requisitos funcionales:

- **RF1. Autenticación y control de acceso**

El sistema debe permitir el inicio de sesión mediante usuario y contraseña, y bloquear la cuenta tras tres intentos fallidos consecutivos.

- **RF2.Gestión de datos (CRUD)**

El sistema debe permitir crear, consultar, actualizar y eliminar registros de usuarios con validación de datos en cada operación.

- **RF3. Procesamiento y cálculo**

El módulo de facturación deberá calcular automáticamente el IVA, descuentos y total de cada pedido según las reglas de negocio vigentes.

- **RF4. Reportes y exportación**

El sistema debe generar reportes de ventas en formato PDF y Excel, filtrables por fecha, producto y vendedor.

- **RF5. Notificaciones y alertas**

El sistema deberá enviar una notificación por correo electrónico al usuario cuando su solicitud cambie de estado en el flujo de aprobación.

Por otro lado, los requisitos no funcionales definen el "cómo lo hace" y opera el sistema, estableciendo los criterios de calidad que determinan su robustez y sostenibilidad. No se limitan a una función específica, sino que abarcan la arquitectura que soporta la información, incluyendo atributos como la seguridad para proteger los datos contra alteraciones accidentales y la confiabilidad para asegurar que la automatización genere siempre resultados consistentes. Estos requisitos evalúan la eficiencia del desempeño, garantizando que el software sea escalable ante grandes volúmenes de datos y mantenga tiempos de respuesta mínimos que optimicen la productividad operativa.

Dentro de los requisitos no funcionales se encuentran:

a) Rendimiento

Velocidad y capacidad de respuesta del sistema.

Ejemplo: "El sistema deberá responder en menos de 2 segundos bajo carga de 1.000 usuarios concurrentes".

b) Seguridad

Protección de datos y acceso no autorizado.

Ejemplo: "Todos los datos en tránsito deben cifrarse con TLS 1.2 o superior. Las contraseñas deben almacenarse con bcrypt".

c) Usabilidad

Facilidad de aprendizaje y eficiencia de uso.

Ejemplo: "Un usuario nuevo debe poder completar el proceso de registro sin ayuda en menos de 3 minutos".

d) Disponibilidad / fiabilidad

Tiempo de actividad y tolerancia a fallos.

Ejemplo: "El sistema tendrá una disponibilidad del 99,9 % mensual (máximo 8,7 h de downtime/año) — SLA".

e) Mantenibilidad

Facilidad para corregir, adaptar y mejorar.

Ejemplo: "La cobertura de pruebas unitarias debe ser ≥ 80 %. La complejidad ciclomática no debe superar 10".

f) Portabilidad

Capacidad de ejecutarse en distintos entornos.

Ejemplo: "La app web debe funcionar correctamente en Chrome, Firefox, Safari y Edge en versiones de los últimos 2 años".

g) Escalabilidad

Capacidad de crecer sin degradar el rendimiento.

Ejemplo: "La arquitectura debe soportar escalar horizontalmente hasta 10 instancias sin reconfiguración manual".

h) Accesibilidad

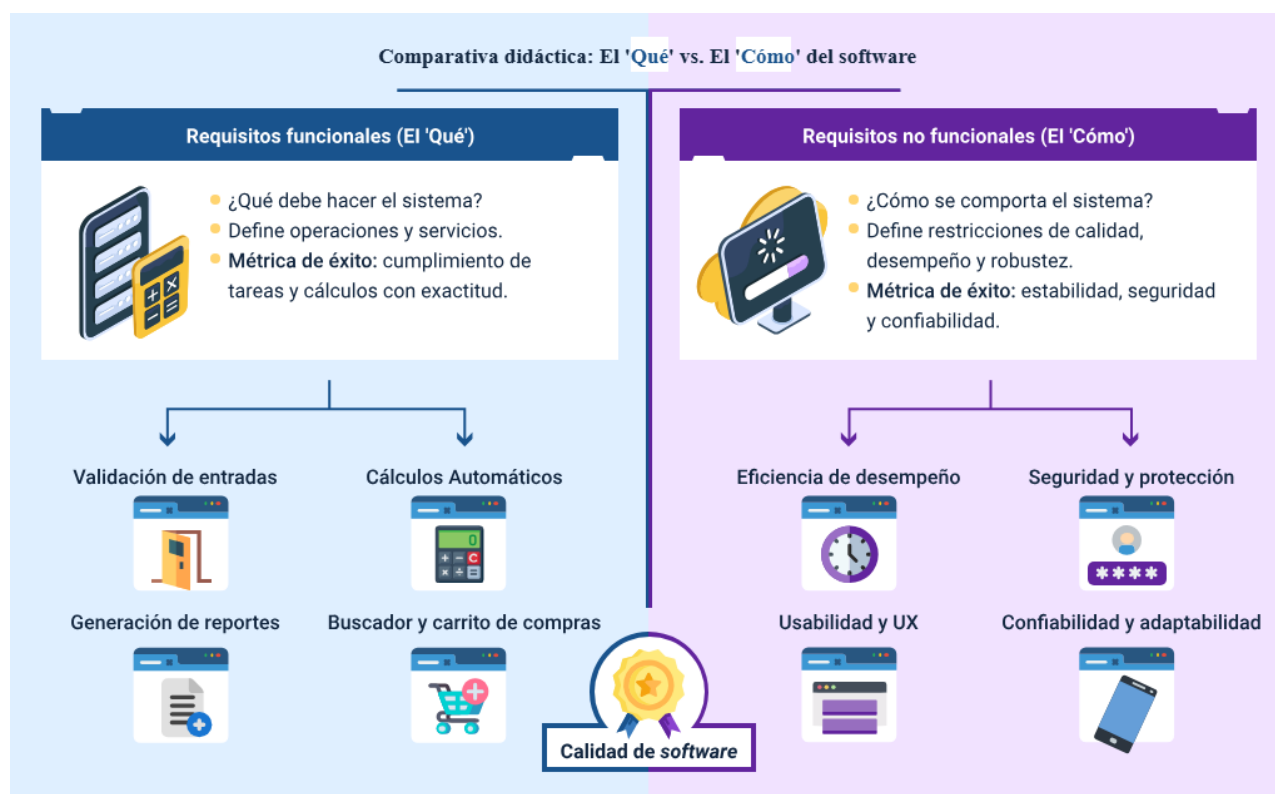
Usable por personas con discapacidades.

Ejemplo: "La interfaz deberá cumplir con el nivel AA de las pautas WCAG 2.1 para garantizar accesibilidad universal".

La evaluación integral de ambos tipos de requisitos permite que el analista no solo detecte fallas funcionales, sino que también identifique hallazgos críticos en la usabilidad o la accesibilidad. Al documentar estas necesidades, se construye una visión de calidad orientada a la excelencia del servicio, donde la simplicidad de la interfaz de usuario permite obtener respuestas clave del negocio sin requerir conocimientos técnicos profundos, asegurando así que el producto final sea fiable, consistente y genere un valor estratégico real para la organización.

La siguiente imagen resume un poco indicado en ambos requisitos:

Figura 8. Comparativa requisitos funcionales y no funcionales



Comparativa requisitos funcionales y no funcionales

Comparativa didáctica: El 'Qué' vs. El 'Cómo' del software

- **Requisitos funcionales (El 'Qué')**
 - ¿Qué debe hacer el sistema?
 - Define operaciones y servicios.
 - Métrica de éxito: cumplimiento de tareas y cálculos con exactitud.
 - Validación de entradas

- Cálculos Automáticos
- Generación de reportes
- Buscador y carrito de compras

Todo esto da Calidad de software

- **Requisitos no funcionales (El 'Cómo')**

- ¿Cómo se comporta el sistema?
- Define restricciones de calidad, desempeño y robustez.
- Métrica de éxito: estabilidad, seguridad y confiabilidad
- Eficiencia de desempeño
- Reducir macro de 4 horas a pocos segundos.
- Seguridad y protección
- Usabilidad y UX
- Generar Informe
- Confiabilidad y adaptabilidad

6.2. Ingeniería de requisitos

Es la disciplina que transforma las necesidades de los interesados en especificaciones técnicas claras para asegurar la calidad del producto final. Este proceso se divide en dos grandes ejes:

- **El análisis de requisitos**

Estudia y valida las funciones que el sistema debe cumplir para responder exactamente a los requerimientos gerenciales.

- **La gestión de requisitos**

Garantiza la trazabilidad bidireccional entre las historias de usuario y el código final para controlar cambios y evitar ambigüedades durante el desarrollo.

En entornos modernos, se adoptan prácticas ágiles para gestionar el flujo de trabajo de manera iterativa e incremental, utilizando el Product Backlog como la fuente única de verdad donde se priorizan las tareas pendientes. El equipo descompone estas necesidades en historias de usuario validadas con criterios de aceptación claros, las cuales se planifican en iteraciones o sprints cortos para entregar incrementos funcionales constantes. Además, herramientas como el storyboard o los prototipos visuales ayudan a capturar el conocimiento del usuario de forma intuitiva antes de iniciar la construcción técnica.

Para que estas prácticas sean efectivas, la estimación de esfuerzo debe basarse en datos históricos y técnicas colaborativas como el Planning Poker para evitar compromisos imposibles de cumplir. Este enfoque ágil permite medir la productividad y la velocidad real del equipo en cada ciclo, facilitando una mejora continua que reduce la deuda técnica y garantiza que el software sea robusto, seguro y confiable.

6.3. Técnicas de elicitación de requisitos

La correcta definición de requisitos es un paso crítico para el éxito de cualquier proyecto de software, ya que permite comprender con precisión las necesidades reales del negocio y de los usuarios. Para lograrlo, el análisis no se apoya en una única fuente de información, sino en un conjunto de técnicas complementarias que facilitan la exploración, comprensión y validación de dichas necesidades. A continuación, se

presentan las principales técnicas de elicitación de requisitos, organizadas según su enfoque y propósito, las cuales permiten capturar tanto información explícita como conocimiento tácito, garantizando soluciones alineadas con la calidad, la innovación y la sostenibilidad del sistema:

- **Técnicas de prospección**

El viaje de la calidad comienza con las técnicas de prospección, que funcionan como una exploración directa del terreno operativo. En este escenario, el analista interactúa con los interesados mediante entrevistas o cuestionarios para identificar tanto los requisitos funcionales (el "qué" debe hacer el sistema) como los no funcionales (el "cómo" debe operar en términos de seguridad o desempeño). Esta fase de diagnóstico es vital, ya que permite establecer el estado de la calidad antes de cualquier intervención técnica, asegurando que la solución final responda fielmente a las necesidades reales del negocio.

- **Técnicas de creatividad**

Cuando las necesidades no son evidentes o se busca transformar radicalmente un servicio, entran en juego las técnicas de creatividad. Estas buscan elevar el estándar de desempeño más allá de los requisitos mínimos, aprovechando oportunidades de optimización que el usuario quizás no ha verbalizado de forma explícita. En lugar de limitarse a corregir fallos del pasado, estas técnicas fomentan un enfoque de mejoramiento continuo e innovación, permitiendo que el equipo visualice soluciones que garanticen la sostenibilidad del software en producción.

- **Técnicas centradas en la documentación**

Por otro lado, las técnicas centradas en la documentación ofrecen una base de conocimiento explícito que es fundamental para la estabilidad y fiabilidad del proyecto. Aquí, el analista se sumerge en manuales, procedimientos previos, registros de lecciones aprendidas y bases de datos existentes para capturar el saber que ya ha sido formalizado por la organización. Esta revisión documental permite construir una trazabilidad sólida, conectando los requisitos históricos con las nuevas especificaciones técnicas para asegurar que la arquitectura resultante sea robusta y consistente.

- **Técnicas de observación**

Finalmente, las técnicas de observación son la herramienta clave para capturar el valioso conocimiento tácito, aquel que reside en las personas y se manifiesta a través de sus habilidades prácticas y hábitos cotidianos. Mediante la observación directa del flujo de trabajo, el analista puede identificar inconsistencias o cuellos de botella que los usuarios suelen omitir en las entrevistas por pura costumbre. Al ver cómo interactúan realmente con las herramientas, se pueden documentar hallazgos críticos de usabilidad o tiempos de respuesta, garantizando que cualquier plan de mejora posterior ataque las causas raíz sistémicas y no simples síntomas superficiales.

6.4. Fuentes de requisitos

El descubrimiento de requisitos comienza con los implicados o interesados (stakeholders). Estas personas son la fuente primaria de información, pues representan las necesidades reales del negocio y las expectativas que el software debe satisfacer. En esta categoría se encuentra desde la gerencia, que busca indicadores

estratégicos en dashboards interactivos, hasta los usuarios operativos que requieren herramientas funcionales y fáciles de usar. Interactuar con ellos permite capturar tanto los requisitos funcionales como el valioso conocimiento tácito experiencias e intuiciones que es fundamental para que la solución sea verdaderamente efectiva en su contexto real de trabajo.

Por otro lado, la documentación existente constituye la fuente de conocimiento explícito por excelencia. Al sumergirse en manuales de usuario, procedimientos técnicos, registros de lecciones aprendidas de proyectos anteriores y reportes de hallazgos, el analista puede construir una trazabilidad sólida de la información. Esta fuente permite comprender la arquitectura que soporta la información actual y ayuda a evitar la repetición de errores pasados, facilitando una transición hacia formatos óptimos y procesos de mejora continúa basados en evidencia objetiva.

Finalmente, los sistemas en operación actúan como una fuente viva de datos técnicos y operativos. Evaluar cómo funcionan las soluciones actuales —ya sean reportes automatizados, macros de Excel o bases de datos— permite realizar un diagnóstico inicial de la calidad del producto y del proceso. Mediante el análisis estático y dinámico de estos sistemas, el analista identifica deudas técnicas, cuellos de botella en los tiempos de respuesta y fallas de seguridad o confiabilidad. Esta observación técnica es la que habilita la formulación de planes de mejora con acciones correctivas y preventivas que garanticen la excelencia del servicio de software a largo plazo.

7. Estadística descriptiva

Es la rama de la estadística que reúne, organiza, resume y presenta un conjunto de datos mediante tablas, gráficas y medidas numéricas, sin extraer conclusiones más allá de los datos observados. Además, permite transformar un conjunto de datos crudos en información comprensible. Es la base del análisis cuantitativo en software (métricas de defectos, cobertura, tiempos de respuesta).

Para aplicar correctamente la estadística descriptiva, es necesario comprender algunos conceptos fundamentales que definen el origen y la naturaleza de los datos analizados. Estos elementos permiten delimitar el alcance del estudio, garantizar la representatividad de la información y asegurar una interpretación adecuada de los resultados:

- **Población:** conjunto total de individuos u objetos que comparten una característica de interés.
- **Muestra:** subconjunto representativo seleccionado de la población para su estudio.
- **Elemento:** cada unidad individual que forma parte de la población o muestra.
- **Característica:** propiedad o atributo medido u observado en cada elemento.

Una vez definidos los elementos básicos del análisis estadístico, el siguiente paso consiste en identificar las variables, ya que son ellas las que representan las características que se miden u observan en cada elemento de la población o muestra. Las variables permiten transformar los datos en información útil, facilitando su

organización, clasificación y análisis mediante las herramientas propias de la estadística descriptiva.

Basado en lo anterior, a continuación, se presentan los principales tipos de variables que se emplean en el estudio y análisis de datos:

a) Variables cualitativas y cuantitativas

Las variables pueden clasificarse según el tipo de información que representan y la forma en que se miden, lo cual orienta el análisis estadístico a realizar. Estas son:

- **Variables cualitativas:** describen cualidades, categorías o atributos que no se expresan numéricamente.

Ejemplo: tipo de defecto (funcional, usabilidad, seguridad), nivel de satisfacción del usuario (bajo, medio, alto), estado de una incidencia (abierta, en proceso, cerrada).

- **Variables cuantitativas:** representan cantidades o valores numéricos que pueden medirse y analizarse matemáticamente.

Ejemplo: número de defectos detectados, tiempo de respuesta del sistema (en segundos), cantidad de usuarios activos, porcentaje de cobertura de pruebas.

b) Variable cualitativa no numérica

Representa categorías, atributos o cualidades que no se expresan mediante números y no permiten operaciones matemáticas. Se clasifica en:

- **Nominal:** no presenta un orden entre sus categorías.

Ejemplos: lógica, interfaz, seguridad, datos.

- **Ordinal:** presenta un orden o jerarquía entre las categorías.

Ejemplo: niveles de madurez: inicial, gestionado, definido, cuantitativamente gestionado y optimizado.

c) Variable cuantitativa numérica

Expresa cantidades medibles. Se pueden realizar operaciones matemáticas con sus valores.

Ejemplo: tiempo de respuesta del sistema medido en segundos, número de errores encontrados en una prueba, cantidad de usuarios registrados o porcentaje de casos de prueba ejecutados.

d) Variables discretas y continuas:

Cada una de ellas representa:

- **Discreta:** valores enteros contables.

Ejemplo: n.º de defectos: 3, 7, 12.

- **Continua:** valores en intervalos.

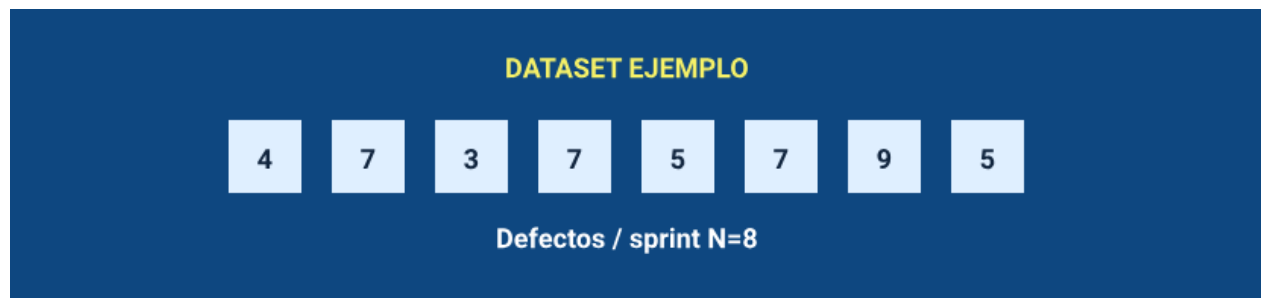
Ejemplo: tiempo de respuesta: 1.24 s.

7.1. Medidas de tendencia central

Para resumir y analizar un conjunto de datos, la estadística descriptiva utiliza indicadores que permiten identificar un valor representativo del comportamiento general de la información. Estas medidas facilitan la interpretación de los datos y

apoyan la toma de decisiones al mostrar el punto central o más típico del conjunto analizado. Se invita a analizar este breve ejemplo:

Figura 9. Ejemplo dataset



Ejemplo dataset

- Dataset ejemplo: 4 7 3 7 5 7 9 5
- Defectos / sprint N=8

A continuación, se presentan las principales medidas de tendencia central:

- **Media:** $\Sigma x_i / n$: suma de todos los valores dividida entre el número de datos. Es la más usada.

Ejemplo:

$$4+7+3+7+5+7+9+5 = 47$$

$$\bar{x} = 47/8 = 5.875$$

- **Media geométrica:** $\sqrt[n]{x_1 \cdot x_2 \cdot \dots \cdot x_n}$: Raíz n-ésima del producto de todos los valores.

Útil para tasas de crecimiento.

Ejemplo:

$$\sqrt[8]{4 \cdot 7 \cdot 3 \cdot 7 \cdot 5 \cdot 7 \cdot 9 \cdot 5}$$

$$G \approx 5.45$$

- **Mediana:** valor central (ordenado). Valor que divide el conjunto ordenado en dos mitades iguales. Robusta ante valores extremos.

Ordenado: 3,4,5,5,7,7,7,9

$$Md = (5+7)/2 = 6$$

- **Moda:** el valor que aparece con mayor frecuencia. Puede haber más de una moda (bimodal, multimodal).

Ejemplo: el 7 aparece 3 veces.

$$Mo = 7$$

Las variables cualitativas se analizan con gráficas de barra o pastel (ejemplo: distribución de defectos por tipo); mientras que las cuantitativas permiten calcular medias, medianas y representarse en histogramas (ejemplo: tiempos de ejecución de pruebas).

Para finalizar, es importante conocer lo que significa cada gráfica de datos y manera de representarse:

a) Gráfico de barras

Se utiliza para comparar valores entre diferentes categorías de forma clara y directa. Cada barra representa una categoría específica, y su altura o longitud es proporcional al valor que se desea analizar, lo que facilita la identificación de diferencias y similitudes.

b) Gráfico pastel (o circular)

Permite visualizar la distribución porcentual de un conjunto de datos. Cada sector del círculo representa una categoría y muestra qué proporción del total le corresponde, siendo útil para análisis de participación o composición.

c) Histograma

Se emplea para analizar la distribución de datos cuantitativos continuos. Los valores se agrupan en intervalos o clases, y las barras muestran la frecuencia de datos en cada rango, lo que ayuda a identificar patrones, dispersión y concentración.

d) Gráfico de líneas

Representa la variación de los datos a lo largo del tiempo o de una secuencia ordenada. Los puntos se conectan mediante líneas, lo que facilita la observación de tendencias, cambios y comportamientos a lo largo del periodo analizado.

¿Cuándo usar cada gráfica?

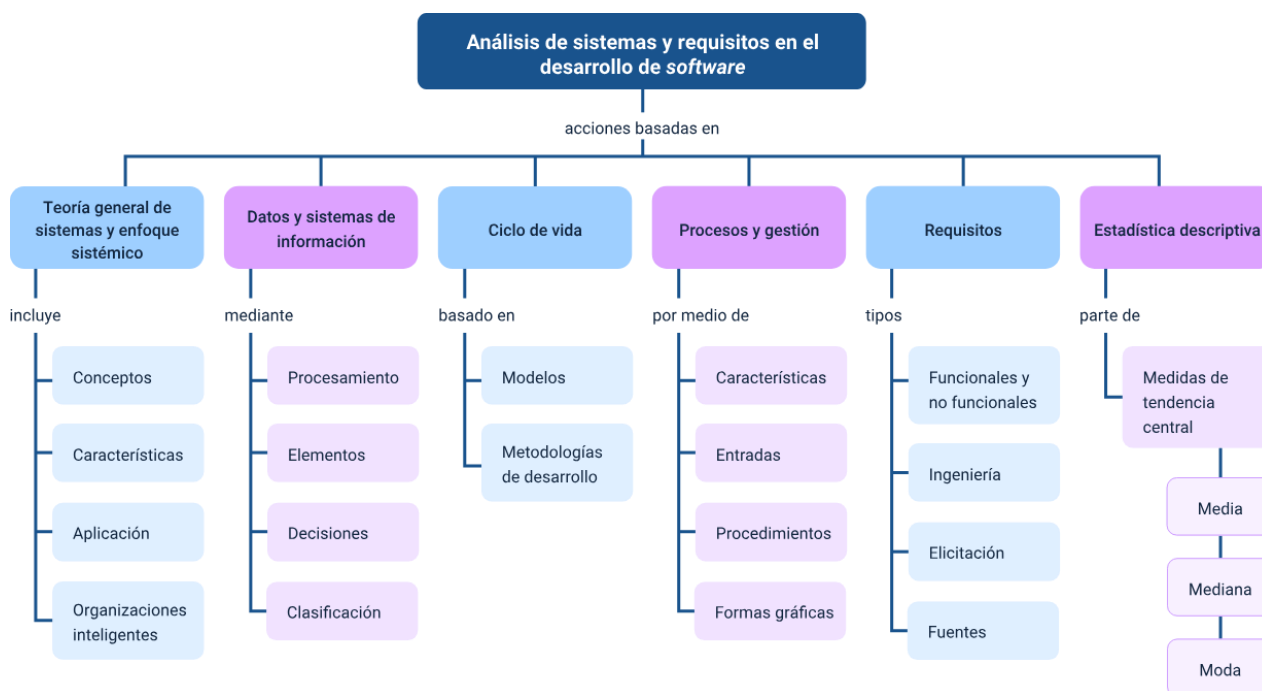
Cada tipo de gráfica se utiliza según el objetivo del análisis: el gráfico de barras es adecuado para comparar categorías, el gráfico pastel para representar proporciones del total, el histograma para analizar la distribución de valores continuos y el gráfico de líneas para observar tendencias a lo largo del tiempo. En pruebas de software, estas

gráficas permiten analizar métricas de QA como la densidad de defectos, los tiempos de ejecución, la cobertura de pruebas y la evolución de la calidad por sprint.

Síntesis

Este componente formativo integra conceptos fundamentales para comprender cómo se estructuran, analizan y gestionan los sistemas de información en contextos organizacionales y tecnológicos. Se abordan principios del enfoque sistémico, la relación entre datos e información y el papel de los sistemas de información como soporte para la gestión y la toma de decisiones.

Asimismo, se analizan elementos clave del desarrollo de software, la definición y gestión de procesos, así como la identificación y tratamiento de requisitos mediante enfoques estructurados y prácticas ágiles. Complementariamente, se incorporan nociones de estadística descriptiva que permiten organizar, interpretar y representar datos mediante medidas e indicadores, fortaleciendo el análisis de información y el apoyo a la evaluación de procesos y soluciones tecnológicas.



Glosario

Ambiente de staging: escenario de pruebas que replica exactamente las condiciones del entorno de producción, utilizado para realizar las validaciones finales antes del despliegue real.

Backlog: listado priorizado de tareas, requisitos o errores que el equipo de desarrollo debe abordar durante el ciclo de vida del proyecto.

Benchmarking: técnica de gestión que consiste en comparar los procesos o métricas de calidad de la propia organización con las mejores prácticas de la industria para identificar oportunidades de mejora.

Build: versión específica del software que se genera tras compilar el código fuente y que está lista para ser sometida a procesos de prueba o ejecución.

Commit: acción de registrar y confirmar un conjunto de cambios en el código fuente dentro de un sistema de control de versiones.

Deuda técnica: costo futuro derivado de elegir soluciones rápidas o "atajos" en el desarrollo en lugar de usar enfoques de mejor calidad que requieren más tiempo.

LMS (Learning Management System): plataforma tecnológica diseñada para la gestión, distribución y control de actividades de formación no presencial o virtual.

Pipeline: conjunto de procesos automatizados que permiten que el código sea probado, integrado y desplegado de manera continua y eficiente

Pruebas de regresión: verificaciones que se realizan tras un cambio o corrección en el software para asegurar que las funcionalidades existentes no se hayan visto afectadas negativamente.

Referencias bibliográficas

Beck, K., et al. (2001). Manifiesto por el Desarrollo Ágil de Software (versión en español). Agile Alliance.

Boehm, B. W. (1988). A Spiral Model of Software Development and Enhancement. IEEE Computer, 21(5), 61–72.

Laudon, K. C. & Laudon, J. P. (2020). Sistemas de información gerencial: Administración de la empresa digital (16.ª ed.). Pearson Educación.

Royce, W. W. (1970). Managing the Development of Large Software Systems. Proceedings of IEEE WESCON.

Senge, P. M. (1990). La quinta disciplina: El arte y la práctica de la organización abierta al aprendizaje. Nueva York: Doubleday.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Responsable Ecosistema de Recursos Educativos Digitales (RED)	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Solanlly Sánchez Melo	Experta temática	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Juan Daniel Polanco Muñoz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Veimar Celis Meléndez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima

Nombre	Cargo	Centro de Formación y Regional
Manuel Felipe Echavarria Orozco	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Ernesto Navarro Jaimes	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Javier Mauricio Oviedo	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima